DEBRE BRHAN UNIVERSITY

COLLEGE OF COMPUTING

DEPARTMENT OF COMPUTER SCIENCE

Individual Assignment-I

Topic: Job Sequencing with Deadlines Algorithmic Strategies using the Greedy Method

Course: Design and Analysis of Algorithms

Submitted By : Beza Tasew ---------- DBU1405586

Submission date: February 13, 2026

Submitted to: Bekele M.

Table of Contents

1. Algorithm Definition and Examples

1.1. Understanding the Problem

In the realm of computational logic, the Job Sequencing with Deadlines problem addresses a common real-world challenge: resource management under time constraints. Imagine a single worker (a processor) who has multiple tasks (jobs) to complete. Each task has a ”use-by date” (deadline) and a ”reward” (profit).

The constraint is that each job takes exactly one unit of time to complete. If you do not finish the job by its deadline, you receive no reward. The objective is not to complete every job, but to select the optimal subset of jobs at the correct times to maximize the total reward.

1.2. Working Principles (The Greedy Logic)

The Greedy Method is an algorithmic paradigm that builds up a solution piece by piece, always choosing the next piece that offers the most immediate benefit. For job sequencing, ”immediate benefit” is defined as the highest profit.

- Profit Prioritization: All jobs are sorted in descending order of profit to ensure high-value tasks are considered first.
- The ”Late Bird” Strategy: For each job, we attempt to schedule it as late as possible (at its deadline). This preserves earlier time slots for jobs that might have much tighter deadlines.
- Conflict Resolution: If the slot at the job’s deadline is occupied, we move backward ($deadline - 1$, $deadline - 2$...) until a free slot is found.

1.3. Illustrative Example & Graphical Trace

Consider a scenario with 5 jobs:

Job ID	Deadline (Units)	Profit ($)
J1	2	100
J2	1	19
J3	2	27
J4	1	25
J5	3	15

Graphical Trace: Maximum Deadline = 3. We have three slots: [1], [2], and [3].

Step	Job	Profit	Deadline	Action	Schedule State
1	J1	100	2	Occupy Slot 2	[E, J1, E]
2	J3	27	2	Slot 2 full; use Slot 1	[J3, J1, E]
3	J4	25	1	Slot 1 full; Discard	[J3, J1, E]
4	J2	19	1	Slot 1 full; Discard	[J3, J1, E]
5	J5	15	3	Occupy Slot 3	[J3, J1, J5]

Result: The optimal sequence is J3 → J1 → J5 with a total profit of 142.

2. Pseudo-code Implementation

The following pseudo-code describes the logic for finding the optimal position for each job within an array of *n* numbers.

```
Algorithm JobSequencing(Jobs, n)
    Input: Array of 'n' Jobs (id, deadline, profit)
    Output: Optimal sequence of Jobs to maximize profit

    1. Sort(Jobs) based on Profit in descending order

    2. Find max_d (Maximum deadline in Jobs)
    3. schedule = array[1...max_d] initialized to NULL
    4. total_profit = 0

    5. for i = 1 to n:
           for j = Jobs[i].deadline down to 1:
               if schedule[j] == NULL:
                   schedule[j] = Jobs[i].id
                   total_profit += Jobs[i].profit
                   break

    6. return schedule, total_profit
```

Listing 1: Job Sequencing Greedy Pseudo-code

3. Time and Space Complexity Analysis

3.1. Time Complexity

- Best-Case $O(n \log n)$: If deadlines are perfectly distributed so that every job immediately finds its slot at *deadline*, the sorting step dominates.
- Worst-Case $O(n^2)$: Occurs when many jobs have the same high deadline. The inner loop scans the schedule backwards for every job.
- Average-Case $O(n^2)$: As the number of jobs *n* increases relative to available slots, colli- sions become frequent.

3.2. Space Complexity

- $O(n)$: The algorithm requires storage for the job list and the result schedule array, which is proportional to n or the maximum deadline .

4. Strengths and Weaknesses

Feature	Greedy Approach	Alternatives (Dynamic Programming)
Simplicity	High; matches human intuition.	More complex; requires defining sub-problems and tables.
Efficiency	Slower for very large n ($O(n^2)$).	Can be optimized to $O(n \times d)$.
Flexibility	Struggles with variable job lengths.	Handles job dependencies and varying lengths better.

5. Proposed Contribution

The ”Smart Pointer” Enhancement (DSU): I propose integrating a Disjoint Set Union (DSU) data structure to optimize the slot search. Instead of a linear backward scan, DSU allows us to find the next available slot in nearly constant time using path compression. Time Complexity Reassessment: This improves the scheduling phase to almost $O(n)$. The total complexity becomes $O(n \log n)$ due to sorting, effectively removing the quadratic bottle- neck.

6. Conclusion

The Job Sequencing with Deadlines problem illustrates a classic optimization challenge in algorithm design, where the goal is to maximize profit under strict time constraints. Using the Greedy Method provides an intuitive and effective solution for many practical cases. By prioritizing high-profit jobs and scheduling them as late as possible, the algorithm efficiently selects an optimal subset of tasks, as demonstrated in the illustrative example.

However, the Greedy Approach has limitations, particularly in flexibility and efficiency for larger datasets or complex job dependencies. Alternative methods like Dynamic Programming can address these challenges but at the cost of increased complexity. Enhancements such as integrating a Disjoint Set Union (DSU) structure can further optimize the Greedy Algorithm, significantly improving scheduling efficiency by reducing the search time for available slots.

Overall, the study of this problem reinforces the importance of understanding algorithmic strategies, trade-offs, and potential optimizations, providing valuable insights for both theoretical and real-world applications in computational problem-solving.

7. References

- Horowitz, E., Sahni, S. (2008). *Fundamentals of Computer Algorithms*. Universities Press.
- Cormen, T. H. (2009). *Introduction to Algorithms*. MIT Press.
- Kleinberg, J., & Tardos, É. (2006). *Algorithm Design*. Pearson Education.